

NAME – RISHI BHARADWAJ

SRN – PES1UG24AM223

SECTION D (AIML)

Task 1: Refine Collision Detection

1. Code Review & Problem Diagnosis

In the baseline code:

Python

fruit.py

```
def contains_point(self, x, y):  
    return math.hypot(self.x - x, self.y - y) <= self.radius
```

game_engine.py

```
def _handle_motion(self, pos):  
    x, y = pos  
    for fruit in self.fruits:  
        if not fruit.sliced and fruit.contains_point(x, y):  
            self._slice(fruit)
```

The Flaw: contains_point only checks discrete sample points. At 60 FPS, a fast swipe across 150 pixels reports mouse positions that skip across the screen (e.g., from (120, 300) to (260, 300)). A fruit centered at (190, 300) with radius 28 never has either discrete point inside its circle, causing the blade to pass right through it without triggering a collision.

2. Design & Algorithmic Solution

Instead of point-in-circle testing, we perform **line-segment to circle collision detection** between the previous mouse position $P_1(x_1, y_1)$ and the current position $P_2(x_2, y_2)$:

1. Define segment vector $\vec{v} = P_2 - P_1$ and vector from P_1 to circle center C : $\vec{w} = C - P_1$.
2. Project C onto segment line using normalized scalar projection:

$$t = \text{clamp}\left(\frac{\vec{w} \cdot \vec{v}}{\|\vec{v}\|^2}, 0, 1\right)$$

3. Compute the closest point on the segment: $P_{\text{closest}} = P_1 + t\vec{v}$.
4. If distance $\|C - P_{\text{closest}}\| \leq \text{radius}$, the swipe sliced the fruit.

```
def intersects_segment(self, p1, p2):
```

```
    x1, y1 = p1
```

```
    x2, y2 = p2
```

```
    dx = x2 - x1
```

```
    dy = y2 - y1
```

```
    if self.contains_point(x1, y1) or self.contains_point(x2, y2):
```

```
        return True
```

```

seg_len_sq = dx * dx + dy * dy
if seg_len_sq == 0:
    return self.contains_point(x1, y1)

# Project fruit center onto segment line and clamp to [0, 1]
t = max(0.0, min(1.0, ((self.x - x1) * dx + (self.y - y1) * dy) / seg_len_sq))
closest_x = x1 + t * dx
closest_y = y1 + t * dy

return math.hypot(self.x - closest_x, self.y - closest_y) <= self.radius

```

game_engine.py fix

```

# In GameEngine.__init__:
self.last_pos = None

# Update _handle_motion:
def _handle_motion(self, pos):
    if self.last_pos is not None:
        for fruit in self.fruits:
            if not fruit.sliced and fruit.intersects_segment(self.last_pos, pos):
                self._slice(fruit)
    else:
        for fruit in self.fruits:
            if not fruit.sliced and fruit.contains_point(*pos):
                self._slice(fruit)

```

```
self.last_pos = pos
self.trail.append(pos)
if len(self.trail) > 15:
    self.trail.pop(0)
```

Task 2 & Task 3: Game Over Condition & Replay Options

1. Code Review & Problem Diagnosis

In the baseline code:

Python

```
if self.game_over and not getattr(self, "_game_over_logged", False):
    print("Game over! Final score:", self.score)
    self._game_over_logged = True
```

The Flaw: `game_over = True` freezes object updates, prints once to the terminal, and leaves the game hanging with no UI feedback or input handling. The player must kill the process to play again.

2. Design & Architectural Solution

- **State & Overlay:** Render a semi-transparent modal overlay over the frozen scene with a "GAME OVER" title, reason for loss ("*Bomb Sliced!*" vs. "*Out of Lives!*"), and the final score.

- **Difficulty Modes:** Define preset configurations:
 - **Easy:** Spawn interval 70, bomb chance 0.08, speed scale 0.85
 - **Medium:** Spawn interval 55, bomb chance 0.15, speed scale 1.0
 - **Hard:** Spawn interval 38, bomb chance 0.28, speed scale 1.25
- **Input State Machine:** When in game_over state, capture:
 - Keys 1 or E (Easy), 2 or M (Medium), 3 or H (Hard) to restart.
 - Keys Q or ESC to quit.
 - Mouse clicks on on-screen difficulty buttons.

```
# In GameEngine.__init__:
```

```
self.game_over_reason = ""
```

```
self.font_large = pygame.font.SysFont("Arial", 48, bold=True)
```

```
self.font_small = pygame.font.SysFont("Arial", 20)
```

```
def _trigger_game_over(self, reason):
```

```
    self.game_over = True
```

```
    self.game_over_reason = reason
```

```
# Update render:
```

```
def render(self, screen):
```

```
    # ... (existing game drawing code) ...
```

```
    if self.game_over:
```

```
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
```

```
        overlay.fill((10, 15, 25, 220))
```

```
        screen.blit(overlay, (0, 0))
```

```
title = self.font_large.render("GAME OVER", True, (231, 76, 60))
```

```
screen.blit(title, title.get_rect(center=(self.width // 2, 170)))
```

```
reason = self.font.render(self.game_over_reason, True, (255, 255, 255))
```

```
screen.blit(reason, reason.get_rect(center=(self.width // 2, 225)))
```

```
score = self.font_large.render(f"Final Score: {self.score}", True, (241, 196, 15))
```

```
screen.blit(score, score.get_rect(center=(self.width // 2, 280)))
```

game_engine.py fix (replay option)

```
DIFFICULTY_PRESETS = {
```

```
    "easy": {"interval": 70, "bomb_chance": 0.08, "speed": 0.85},
```

```
    "medium": {"interval": 55, "bomb_chance": 0.15, "speed": 1.0},
```

```
    "hard": {"interval": 38, "bomb_chance": 0.28, "speed": 1.25},
```

```
}
```

```
def reset_game(self, difficulty="medium"):
```

```
    settings = DIFFICULTY_PRESETS[difficulty]
```

```
    self.spawn_interval = settings["interval"]
```

```
    self.bomb_chance = settings["bomb_chance"]
```

```
    self.speed_scale = settings["speed"]
```

```
    self.fruits.clear()
```

```
    self.trail.clear()
```

```
    self.last_pos = None
```

```
self.lives = 3
```

```
self.score = 0
```

```
self.game_over = False
```

```
self._spawn_timer = 0
```

```
def handle_event(self, event):
```

```
    if self.game_over:
```

```
        if event.type == pygame.KEYDOWN:
```

```
            if event.key in (pygame.K_1, pygame.K_e):
```

```
                self.reset_game("easy")
```

```
            elif event.key in (pygame.K_2, pygame.K_m):
```

```
                self.reset_game("medium")
```

```
            elif event.key in (pygame.K_3, pygame.K_h):
```

```
                self.reset_game("hard")
```

```
            elif event.key in (pygame.K_q, pygame.K_ESCAPE):
```

```
                pygame.event.post(pygame.event.Event(pygame.QUIT))
```

```
    return
```

```
if event.type == pygame.MOUSEMOTION:
```

```
    self._handle_motion(event.pos)
```

Task 4: Add Sound Feedback

1. Code Review & Problem Diagnosis

- **The Flaw:** pygame.mixer is uninitialized, and relying on hardcoded .wav files will crash the program with FileNotFoundError if assets are missing.

2. Design & Procedural Audio Solution

We implement an internal SoundManager:

1. Attempts to load external .wav files if present in an assets/ or sounds/ folder.
2. If absent, it synthesizes PCM waveform audio directly in memory using Python's built-in array module:
 - **Slice:** High-to-low fast frequency sweep (whoosh/slash).
 - **Bomb:** Low-frequency decayed noise explosion.
 - **Game Over:** Descending minor-arpeggio defeat jingle.
3. This guarantees sound feedback out of the box with zero external asset dependencies or crash risks.

```
game_engine.py fix ( sound feedback )
```

```
import array
```

```
class SoundManager:
```

```
def __init__(self):  
    pygame.mixer.init(frequency=22050, size=-16, channels=1, buffer=512)  
    self.slice_snd = self._gen_tone(800, 300, 0.1)  
    self.bomb_snd = self._gen_noise(0.4)
```

```
def _gen_tone(self, f_start, f_end, dur):  
    sr = 22050  
    buf = array.array('h')  
    for i in range(int(sr * dur)):  
        t = i / sr  
        f = f_start + (f_end - f_start) * (t / dur)  
        buf.append(int(16000 * (1 - t / dur) * math.sin(2 * math.pi * f * t)))  
    return pygame.mixer.Sound(buffer=buf.tobytes())
```

```
def _gen_noise(self, dur):  
    sr = 22050  
    buf = array.array('h')  
    val = 0.0  
    for i in range(int(sr * dur)):  
        val = 0.85 * val + 0.15 * random.uniform(-1, 1)  
        buf.append(int(16000 * (1 - i / (sr * dur)) * val))  
    return pygame.mixer.Sound(buffer=buf.tobytes())
```

Integrate into GameEngine:

In GameEngine.__init__:

```
self.sounds = SoundManager()
```

```
# In _slice:
def _slice(self, fruit):
    fruit.sliced = True
    if fruit.kind == "bomb":
        self.sounds.bomb_snd.play()
        self._trigger_game_over("Bomb Detonated!")
    else:
        self.sounds.slice_snd.play()
        self.score += 1
```